UNIVERSITY OF INFORMATION TECHNOLOGY AND SCIENCES

SOFTWARE TESTING & QUALITY ASSURANCE · MAPPED CO: CO3

SOFTWARE TESTING REPORT

For

UniQuery — “Ask. Explore. Choose.”

A Smart University Information and Admission Assistant

Prepared By:

Israt Jahan — 0432320005101130

Md. Mehedi Hasan — 0432320005101082

Course: Software Project Design & Development (CSE 416)
Academic Session: Autumn 2026
Test Environment: React 19.2 · Vite 8.3 · Vitest 4.1 · React Testing Library · V8 Coverage

Prepared For:

Dr. Mahfida Amjad Dipa

Senior Teacher · Department of CSE
University of Information Technology and Sciences (UITS)

GitHub Project Source: github.com/Keya2323/Software-Project-Design-Development-CSE-416/tree/main/uniquery
Automated Test Suite: .../uniquery/src/__tests__ | **Defect Fixes Commit:** Commit 75537ec

Date of Submission: 10 November 2026

1. Objective & Testing Strategy Overview

The objective of this testing report (mapped to Course Outcome **CO3**) is to design, execute, refine, and verify an automated software testing strategy for **UniQuery (“Ask. Explore. Choose.”)** — a React 19 and Vite 8 web application that helps admission-seeking students in Bangladesh query official admission rules, evaluate GPA-based eligibility, compare universities side-by-side, and track admission circular deadlines.

Following the course testing guidelines, all tests are structured using the **AAA (Arrange, Act, Assert)** pattern and combine **Equivalence Partitioning (EP)**, **Boundary Value Analysis (BVA)**, **Edge-Case Refinement**, and **End-to-End Component Integration Testing**. Three core automated tests (comprising 9 granular test cases, `TC_01`–`TC_09`) were selected to represent UniQuery’s three architectural stages:

3 Suites (9 TCs)	4 Defects	9 / 9 (100%)	70.87%
CORE TESTS EXECUTED	REVEALED & FIXED	FINAL PASS RATE	BRANCH COVERAGE (V8)

Submitted Code & Test Suite Links (GitHub Repository):
• **Automated Test Files:** https://github.com/Keya2323/Software-Project-Design-Development-CSE-416/tree/main/uniquery/src/__tests__
• **Defect Fix & Verification Diff:** https://github.com/Keya2323/Software-Project-Design-Development-CSE-416/commit/75537ec

Test No.	Target Module & File	Testing Technique	Why Suitable for UniQuery
Test 1 (`TC_01–TC_03`)	**ASK Module** `src/data/chatKnowledge.js` (`getBotResponse()`)	Unit Testing, Equivalence Partitioning & Edge-Case Token Collision	Verifies that the AI chatbot returns verified seat counts, GPA rules, profile-personalized CSE guidance, and avoids false keyword collisions or null-input crashes.
Test 2 (`TC_04–TC_06`)	**EXPLORE Module** `EligibilityCalculatorModal.jsx`	Boundary Value Analysis (BVA) & Equivalence Partitioning	Validates exact GPA cutoffs (e.g., Combined GPA `8.00` vs `7.99` for DU, STEM GP sum `14.5` for BUET) and Science vs non-Science academic group rules.
Test 3 (`TC_07–TC_09`)	**CHOOSE & NOTICES** `App.jsx`, `CompareView`, `ExploreWizard`, `NoticesView`	Component Integration & End-to-End User Flow Testing	Verifies cross-component state propagation: max-4 university comparison cap, wizard profile recalculation flow, and admission notice status filtering & modal view.

2. Test Scenarios & Structured Test Cases Table

Table 2 documents all 9 test cases across the **3 Core Tests** following the standard Test Case specification format (Pre-conditions, Test Steps, Expected Result, Actual Result before and after defect resolution, and final Pass/Fail status).

TC ID	Test Scenario	Pre-conditions	Test Steps	Expected Result	Actual Result	Status
TEST 1: Unit Testing — Conversational AI Knowledge Engine (ASK Module · chatKnowledge.js)						
TC_01	Valid University & Policy Queries	`getBotResponse` module loaded	1. Pass BUET GPA query 2. Pass 2nd-timer query 3. Pass Medical, DU, CKRUET & GST queries	Returns exact seats (BUET 1,305; MBBS 5,380), matching `universityIds`, and official circular source	Returned exact guidelines, IDs, and UGC/portal citations	Pass
TC_02	Profile Context Personalization Toggle	Student profile: SSC `5.00`, HSC `4.92`, Science	1. Call CSE query with `profileContextActive=true` 2. Call same query with `false`	Active mode embeds SSC `5` & HSC `4.92`; muted mode returns generic CSE university list	Personalized GPA injected when active; omitted when muted	Pass
TC_03	Substring Collision & Null Input Edge Case	`getBotResponse` module loaded	1. Query "What are the tuition fees for undergraduate education?" 2. Call `getBotResponse(null)`	Matches Tuition overview (not DU via `"du"` in education) and handles `null` safely without crash	Initial: Failed (matched DU & threw `TypeError`). Post-fix: Matched Tuition & safe fallback	Pass Fixed DEF-01
TEST 2: Boundary Value & Equivalence Partitioning — Instant Eligibility Calculator (EXPLORE Module)						
TC_04	Upper Valid Partition (Golden GPA 10.00)	Calculator modal open; Science group; STEM 15.0	1. Enter SSC GPA = `5.00` 2. Enter HSC GPA = `5.00`	Displays `10.00 / 10.00`, "Eligible for 5 of 5 Tracks", and 50–100% Merit Waiver note	Displayed `10.00`, `5 of 5 Tracks`, and waiver eligibility	Pass
TC_05	Boundary Value Analysis (8.00 vs 7.99 & STEM GP)	Calculator modal open; Science group	1. Set SSC `4.00`, HSC `4.00` (Total `8.00`) 2. Set Math GP `4.0` (STEM `14.0`) 3. Set SSC `3.99` (Total `7.99`)	At `8.00`: DU eligible (4/5 tracks). At STEM `14.0`: BUET fails (<14.5). At `7.99`: DU fails (<8.00), leaving 2/5 tracks	Boundary transitions triggered exact track counts (4/5 → 2/5) and threshold messages	Pass
TC_06	Invalid Discipline Partition (Commerce Group)	Calculator modal open; GPA `5.00 + 4.83`	1. Select Discipline = `Commerce` 2. Inspect BUET, DU KA, and Medical cards	Marks 2 of 5 tracks eligible; Science-only tracks show non-Science rejection reason	Initial: Failed (ineligible card still said "Fully eligible..."). Post-fix: Shows Science requirement	Pass Fixed DEF-02

TC ID	Test Scenario	Pre-conditions	Test Steps	Expected Result	Actual Result	Status
TEST 3: Component Integration & End-to-End Flows (CHOOSE & NOTICES Modules · App.jsx)						
TC_07	Compare Matrix Max-4 University Limit	App loaded with 3 default compared unis (BUET, DU, RU)	1. Open Explore Results 2. Add 4th university (CU) 3. Click Add to Compare on 5th (KUET) & open Compare	4th university added (`4 of 4 selected`); 5th blocked with warning toast; diff highlight toggles	Warning toast shown on 5th click; matrix locked at `4 of 4 selected`	**Pass**
TC_08	Explore Wizard Profile Recalculation Flow	User is on Explore Results view	1. Click "Recalculate Wizard" 2. Change HSC GPA to `4.50` 3. Click "Check Eligibility"	Opens `ExploreWizard`, updates profile state in `App`, and renders `4.50` on Results strip	Initial: Failed (prop mismatch prevented open/submit). Post-fix: Updated HSC GPA to `4.50`	**Pass** **Fixed DEF-03**
TC_09	Notices Status Filter & Official Modal Flow	User navigates to Admission Notices view	1. Select Status = "Closing Soon" 2. Click "View Notice" on BUET 3. Close modal & click "Clear"	Displays closing-soon notices, opens Official Notice Details modal, and resets filters cleanly	Initial: Failed (0 notices due to kebab-case mismatch & missing modal prop). Post-fix: Modal & filter pass	**Pass** **Fixed DEF-04**

3. Automated Test Implementation (AAA Pattern)

Every automated test in uniquery/src/__tests__/ strictly follows the **Arrange – Act – Assert (AAA)** structure taught in the lab handout: setting up inputs/DOM state in Arrange, invoking the target function or user event in Act, and verifying return values or rendered UI state in Assert.

3.1 Test 1 Excerpt — Unit Test using AAA Pattern (test1_askAiKnowledge.test.js)

```
test('TC_02: Applies student profile GPA personalization when profileContextActive is enabled
vs disabled', () => {
  // 1. Arrange: Define input query and mock student profile state
  const query = 'Which universities offer CSE for Science students?';
  const studentProfile = { sscGpa: 5.00, hscGpa: 4.92, academicGroup: 'Science' };

  // 2. Act: Invoke system under test with personalization ON and OFF
  const personalizedResponse = getBotResponse(query, true, studentProfile);
  const genericResponse = getBotResponse(query, false, studentProfile);

  // 3. Assert: Verify dynamic GPA interpolation vs generic response
  expect(personalizedResponse.text).toContain('HSC GPA 4.92');
  expect(personalizedResponse.universityIds).toEqual(['buet', 'du', 'kuet', 'nsu', 'bracu']);
  expect(genericResponse.text).not.toContain('4.92');
});
```

3.2 Test 2 Excerpt — Boundary Value Analysis using AAA Pattern (test2_eligibilityCalculator.test.jsx)

```
test('TC_05: Boundary Value Analysis at Dhaka University threshold (Combined GPA 8.00 vs
7.99)', () => {
  // 1. Arrange: Render Instant Eligibility Calculator Modal
  render(<EligibilityCalculatorModal isOpen={true} onClose={vi.fn()}
onExploreMatches={vi.fn()} />);
  const [sscInput, hscInput] = screen.getAllByRole('spinbutton');

  // 2. Act (At Exact Boundary = 8.00) & 3. Assert
  fireEvent.change(sscInput, { target: { value: '4.00' } });
  fireEvent.change(hscInput, { target: { value: '4.00' } });
  expect(screen.getByText('8.00')).toBeInTheDocument();
  expect(screen.getByText('Eligible for 4 of 5 Tracks')).toBeInTheDocument();

  // 2. Act (Just Below Boundary = 7.99) & 3. Assert
  fireEvent.change(sscInput, { target: { value: '3.99' } });
  expect(screen.getByText('7.99')).toBeInTheDocument();
  expect(screen.getByText('Eligible for 2 of 5 Tracks')).toBeInTheDocument();
});
```

3.3 Test 3 Excerpt — End-to-End Integration Flow using AAA Pattern (test3_integrationFlows.test.jsx)

```
test('TC_07: Enforces maximum 4 universities in Comparison Matrix and shows warning toast on
5th selection', () => {
  // 1. Arrange: Mount full UniQuery App (initial compareList = ['buet', 'du', 'ru'])
  render(<App />);
  fireEvent.click(screen.getByRole('button', { name: /View All 50\+ Universities/i }));

  // 2. Act: Add 4th university, then attempt to add a 5th university
  fireEvent.click(screen.getAllByRole('button', { name: /Add to Compare/i })[0]);
  fireEvent.click(screen.getAllByRole('button', { name: /Add to Compare/i })[0]);

  // 3. Assert: Verify toast guard message and 4-slot matrix boundary
  expect(screen.getByText(/Maximum 4 universities can be compared at once/
i)).toBeInTheDocument();
  fireEvent.click(screen.getByRole('button', { name: /Side-by-Side Comparison Matrix/i }));
  expect(screen.getByText('4 of 4 selected')).toBeInTheDocument();
});
```

4. Initial Execution, Defects Revealed & Fixes Applied

Executing the suite against the initial repository baseline (commit 89e3a50) produced **5 passing tests and 4 failing tests**, exposing four genuine functional defects across UniQuery's keyword matching, eligibility messaging, and parent-child React prop contracts (subsequently resolved in commit 75537ec):

```
 RUN  v4.1.12 /uniquery  (Initial Run — Before Defect Fixes, Commit 89e3a50)
 ❯ src/__tests__/test1_askAiKnowledge.test.js (3 tests | 1 failed)
   ✓ TC_01  ✓ TC_02  × TC_03: AssertionError: expected 'University of Dhaka Science Unit...'
to contain 'Real Tuition & Cost Overview'
 ❯ src/__tests__/test2_eligibilityCalculator.test.jsx (3 tests | 1 failed)
   ✓ TC_04  ✓ TC_05  × TC_06: Found <p>Fully eligible to apply for preliminary screening!</p>
on ineligible Commerce profile
 ❯ src/__tests__/test3_integrationFlows.test.jsx (3 tests | 2 failed)
   ✓ TC_07  × TC_08 (Wizard did not open)  × TC_09 (0 notices returned for "Closing Soon")

 Test Files  3 failed (3)  |  Tests  4 failed | 5 passed (9)
```

Defect ID	Location	Root Cause Revealed by Test	Fix Implemented & Verified
DEF-01 (Revealed by `TC_03`)	`src/data/chatKnowledge.js` (Lines 15, 42)	`q.includes("du")` matched the substring `"du"` inside English words like "undergraduate", "education", and "graduation", routing tuition & scholarship queries to Dhaka University. Passing `null` also threw `TypeError`.	Replaced substring check with word-boundary regex `/\bdu\b/.test(q)` and normalized input safely via `String(query ?? "").toLowerCase().trim()`.
DEF-02 (Revealed by `TC_06`)	`EligibilityCalculatorModal.jsx` (Lines 39–66)	While `status` checked `group === 'Science'`, the `reason` ternary omitted the group check—displaying "Fully eligible..." underneath a red `REQUIREMENTS NOT MET` badge for Commerce/Arts students.	Added explicit `group !== 'Science'` guard first in the `reason` expressions for BUET, DU Science Unit (KA), and Medical (MBBS).
DEF-03 (Revealed by `TC_08`)	`src/App.jsx` (Lines 193–217)	`App.jsx` passed `onOpenWizard` to `ExploreResults` (which expected `onEditProfile`) and `onSaveProfile` to `ExploreWizard` (which invoked `onUpdateProfile` & `onCompleteStep`).	Wired `onEditProfile` on `ExploreResults` and passed `onUpdateProfile` & `onCompleteStep` handlers to `ExploreWizard` in `App.jsx`.
DEF-04 (Revealed by `TC_09`)	`NoticesView.jsx` (Line 24) & `App.jsx` (Line 248)	Dropdown values (`"Closing Soon"`, `"Active"`) failed strict equality against `noticesData` status values (`"closing-soon"`, `"active"`). Also, `App.jsx` omitted `onOpenNotice` and passed `initialModalNoticeId` instead of `activeModalNoticeId`.	Normalized status strings (`toLowerCase().replace(/\s+/g, '-')`) in `NoticesView.jsx` and passed `activeModalNoticeId` + `onOpenNotice` in `App.jsx`.

5. Post-Fix Re-Run, Coverage Report & Gap Analysis

After fixing all four defects (commit 75537ec), the complete test suite and production Vite build were re-executed. All **3 Test Suites (9/9 Test Cases) passed with 0 errors**:

```
RUN  v4.1.12 /uniquery  (Post-Fix Verification Run with V8 Coverage — Commit 75537ec)
✓ src/__tests__/test1_askAiKnowledge.test.js (3 tests) — 7ms
✓ src/__tests__/test2_eligibilityCalculator.test.jsx (3 tests) — 730ms
✓ src/__tests__/test3_integrationFlows.test.jsx (3 tests) — 4381ms

Test Files  3 passed (3)  |  Tests  9 passed (9)  |  Build: vite build ✓ built in 0.44s
```

Source File Covered	% Statements	% Branches	% Functions	% Lines	Verdict
`src/data/chatKnowledge.js` (Test 1)	**100.00%**	**95.65%**	**100.00%**	**100.00%**	**Excellent**
`EligibilityCalculatorModal.jsx` (Test 2)	**76.66%**	**80.00%**	53.84%	**75.00%**	**Strong**
`NoticesView.jsx` (Test 3)	**77.77%**	**71.42%**	66.66%	**81.08%**	**Strong**
`CompareView.jsx` (Test 3)	60.86%	**71.71%**	59.61%	61.90%	**Good**
`App.jsx`, `ExploreResults` & `ExploreWizard`	42.70%	**59.62%**	28.01%	46.30%	**Core Paths**
Aggregate Across Tested Modules	**55.86%**	**70.87%**	**42.92%**	**58.26%**	**Justified**

Coverage Justification & Defect Gap Analysis (Step 3 & Step 4):

• **Why Coverage is Justified:** Core decision logic achieved **100% line coverage** in `chatKnowledge.js` and **71%–80% branch coverage** across `EligibilityCalculatorModal`, `CompareView`, and `NoticesView`. Remaining uncovered lines in `App.jsx` and secondary modals correspond to static footer links, keyboard shortcut palettes (`Cmd+K`), and simulated clipboard/print handlers rather than admission business rules.

• **Residual Defect Categories Identified:** Static UI tests in JSDOM do not cover real browser CSS layout responsiveness, external link availability (e.g., official university admission portals), or multi-tab persistence (`localStorage`), which represent recommended next steps for Playwright E2E testing.